xlq: A Safe-Write Boundary for LLM Agents Editing Spreadsheets

the xlq authors

2026-07-04

Contents

1 xlq: A Safe-Write Boundary for LLM Agents Editing Spreadsheets

An Enforced Fidelity Property, an Oracle-Gated Write Path, and an Adversarial Confinement Trial

Authors omitted for review.

1.1 Abstract

Given the same one-cell edit on the same workbook, a status-quo LLM agent using the standard Python library produced a corrupt, non-reloadable file — two charts and a pivot table stripped of their relationships, 13 of 50 package parts dropped, only 1 of 50 left byte-identical — and *reported success*, having read the raw XML and confirmed its cell had landed; an agent confined to the boundary presented here made the identical edit with charts and pivot byte-identical, 48 of 50 parts untouched, the file reloadable, and a signed receipt. This is the corruption of Anthropic's own shipped spreadsheet skill (issue #22044), reproduced live under a real agent, and prevented. The boundary is xlq, and its core is a *surgical* write primitive with a fidelity property that holds by construction and is enforced on every write: after an edit, every OOXML part that does not contain a changed cell is byte-identical to the input, because the writer copies those parts verbatim and re-serializes only the sheet parts an operation touches — and because every apply byte-diffs the un-edited parts before it commits and aborts on any change — a property that openpyxl, LibreOffice, and even the engine's own whole-file writer all fail, each rewriting the entire container. The write is wrapped in a transactional envelope: a `base_hash` precondition under an advisory lock, a `--dry-run` that predicts affected cells and new formula errors, a proof-carrying commit that re-loads its own output, verifies every predicted cell landed, and confirms every non-edited part survived byte-identical before it commits, a write-reliability gate that refuses to persist any engine-computed cache whose formula uses a function our own differential oracle found divergent from Excel, an atomic revision swap, and an append-only hash-chained receipt journal. We define the enforcement claim precisely — an agent whose harness confines its workbook writes to xlq cannot reach the #22044 failure mode, while an agent handed a raw shell can, which is the harness's responsibility and not xlq's — matching how the 2025–26 agent-enforcement literature scopes its own boundaries. Underneath, the boundary reads a workbook through a privacy-safe structural census (999 B versus a 7.24 MB full-cell dump on a ~100k-formula file), reports its own per-artifact evaluability as three honest numbers (522 of 522 catalog functions recognized, 505 locally evaluable, 17 policy-limited), and rests on a formula engine validated by a documentation-arbitered differential oracle over 1,659 cases; an independent financial cross-check confirms two of the oracle's Treasury-bill verdicts and *downgrades* a third, which we report as a strength of the method rather than hide. We then carry the same surgical discipline to the operation this work had named its own key open problem — *structural edits*, row and column insert and delete, where fidelity and correctness genuinely conflict because a correct edit *must* rewrite the reference-bearing parts a chart or cross-sheet formula depends on: a total reference-shift algebra applied uniformly across every reference-bearing OOXML part under a machine-checkable minimal-patch invariant, so the only bytes that change are reference coordinates and physically inserted rows, with shared-formula expansion and a never-silently-wrong refusal discipline — in a controlled trial xlq shifts 6 of 6 reference classes correctly where openpyxl's `insert_rows` silently corrupts all six, and across 231 real workbooks 78.8% are safely edited and 21.2% refused with a truthful reason, none silently corrupted, an engine-free round-trip oracle passing 182 of 182. All artifacts are reproducible.

1.2 1. Introduction

Anthropic's own Claude Code shipped a spreadsheet skill that quietly destroyed the files it edited (issue #22044): asked to change a few cells in a financial model, the agent completed the requested edit and reported success, but the workbook it saved had lost its charts, pivot tables, and macros — stripped by openpyxl, the standard Python library the skill used to write the file back to disk. The loss was invisible at exactly the moment it mattered: the agent's own report, the diff a reviewer would skim, and the cells the

task touched all looked correct. When an LLM agent edits a spreadsheet, its decisions do not sit in a separate log — they become the cells the file's owner is accountable for [Panko 2008; Hermans and Murphy-Hill 2015]. The agent had no safe way to write the file. Its only primitive was to load the whole workbook into a library that models a subset of the format, mutate a cell, and re-serialize the entire container — discarding everything the library does not model as a side effect of saving.

An earlier version of this work (§10) diagnosed that failure and built a read-only inspector that could *observe* the damage but not prevent it: a boundary that cannot write cannot corrupt, but it also cannot let an agent do the job it was asked to do. That is not an enforcement boundary; it is an abstention. This paper builds the missing primitive. The contribution is a *surgical transactional write* — a way for an agent to change a cell that rewrites only the sheet parts an edit actually touches and copies every other part of the file byte-for-byte — together with the transactional envelope (precondition, dry-run, atomic revision, receipt) that makes each edit previewable, attributable, and reversible, and the threat model that says precisely whom it protects against and whom it does not. The property that makes it a boundary rather than a heuristic holds by construction and is checked at write time, and is the spine of the paper:

> **Fidelity property.** After `xlq apply`, every OOXML part that does not contain a cell changed by the patch is byte-identical to the corresponding input part.

Charts, pivot caches, VBA projects, drawings, styles, theme, and external links are preserved not because the writer is careful with them but because it never touches them — it streams them from the input zip to the output zip unchanged. This is exactly the property openpyxl, LibreOffice, and IronCalc's own `save_to_xlsx` all fail, because each deserializes the model it understands and re-serializes the whole package (§3). It directly closes the #22044 harm class.

The central evidence is interventional, not static — a demonstration-scale case study, not a rate. We ran a real LLM agent on the same edit task, on the same workbook, changing only the tool it was given to write the file. Through the status-quo openpyxl path the agent produced a workbook that will not reopen — charts and pivot stripped of their relationships, 13 of 50 parts dropped, 1 of 50 byte-identical — and reported success anyway, having read the raw sheet XML and seen its target cell written; through xlq the agent made the identical edit with the charts and pivot byte-identical, 48 of 50 parts untouched, the file reloadable, and a hash-chained receipt on disk (§5). The agent, prompt, input file, and environment were held fixed; only the write tool varied, so in this pair of trials the difference in outcome is attributable to the tool boundary. In the trial, an agent confined to xlq could not commit the #22044 corruption *even when it tried the identical edit* — a difference that is mechanical, not a matter of the agent's care.

The rest of the boundary supports that write path and is largely carried over from the read-only system, now in service of a tool that writes. The workbook is read through a privacy-safe structural census, so an agent sees a file's structure without its contents at a fraction of the token cost of dumping cells (999 bytes versus a 7,239,303-byte full-cell dump on a ~100k-formula workbook, a 7,246.5× reduction). The runtime reports its own per-file evaluability as three honest numbers rather than one — 522 of 522 catalog functions recognized, 505 locally evaluable, 17 returning the exact error literal desktop Excel returns — and the dry-run degrades that flag when a volatile or unsupported function lies in an edit's dependency graph, so a write whose predicted values cannot be trusted is refused rather than shipped. The formula engine that powers both the dry-run prediction and the differential comparisons is validated by a documentation-arbitered differential oracle over 1,659 hand-authored cases; a new independent financial cross-check (§8) confirms two of its Treasury-bill verdicts and downgrades a third, which we report openly because the cross-check caught one of our own calls.

This paper makes six contributions. **First**, the surgical write primitive and its fidelity property — enforced and *checked per apply*, each write re-loading its own output as an on-disk `.xlsx` to confirm every predicted cell landed *and* byte-diffing every non-edited OOXML part against the input, aborting with `fidelity_violation` if any un-edited part changed, so the property is checked at write time rather than only asserted by construction — with per-file preservation results on four real workbooks carrying charts, pivots, and VBA (§3). **Second**, an interventional case study — n = 2 tasks, one model, one trajectory per arm — that reproduces the #22044 corruption in a single observed instance under a status-quo agent and shows an xlq-confined agent mechanically unable to commit it, including the false-success failure where the agent believed it had

succeeded (§5); this is demonstration-scale evidence that the harm and its prevention exist, not a rate or a generality claim. **Third**, an explicit threat model whose non-bypassability we tested adversarially in a single adversarial-agent trial rather than only argued: an adversarial agent inside a confined harness could not forge a valid-but-wrong workbook by any route but xlq, surfaced a blunt-destruction hole, and had it closed by a read-only authoritative store and a receipt-gated broker — the boundary confines an agent routed through it, not one handed raw shell (§6). One trial that found and closed a real hole is evidence, not a proof. **Fourth**, the format-aware reading layer — the privacy-safe census and the three-number coverage taxonomy — that lets a write be previewed and its trustworthiness declared per file (§7). **Fifth**, a documentation-arbitered differential oracle for a domain with no mechanized specification, now cross-checked against an independent financial reference that retires the circularity objection for two verdicts and honestly downgrades a third — and whose confusion matrix is now *load-bearing for write safety*, the write refusing to persist any engine-computed cache whose formula uses an oracle-divergent function (§8), packaged with the rest as AXLE-bench, whose fidelity axis measures the dimension current agent benchmarks omit (§9). **Sixth**, a *surgical structural-edit* primitive — row and column insert and delete — that closes the operation this paper's own earlier scope named as its key open problem: a total reference-shift algebra (a faithful re-encoding of documented Excel semantics, and itself *not* billed as the novelty) applied *uniformly* across every reference-bearing OOXML part — sheet and cross-sheet formulas, defined names, chart data refs, pivot sources, and the `sqref` and formula bodies of merged cells, conditional formatting, and data validation — under a machine-checkable *minimal-patch invariant* in which the only bytes that may differ are reference coordinates and physically inserted or removed rows, with shared-formula expansion for the dominant real-world construct and a *never-silently-wrong* discipline that either shifts a construct correctly or refuses it with a truthful reason; in a controlled experiment xlq shifts 6 of 6 reference classes to their documented target while openpyxl's `insert_rows` corrupts all 6 silently, and across a 231-workbook real corpus 78.8% are safely edited (up from 22.5% before expansion) and 21.2% refused, none silently corrupted, with an engine-free round-trip oracle passing 182 of 182 (§4). We are explicit about what we did not invent: receipts, transactional isolation, and differential testing are established [Notarized Agents 2026; Fault-Tolerant Sandboxing 2025; Yang et al. 2011]; the contribution is folding the artifact's *format fidelity* into an agent write boundary — surgically, with an enforced-and-checked fidelity property, and demonstrated under a real agent — for a legacy artifact class where no safe write primitive previously existed.

Scope, stated honestly. This is a tool-and-systems contribution at demonstration scale, and we mark its edges rather than let strong verbs outrun the evidence. What is built and tested: (a) a safe-write primitive for *in-place cell and formula edits*, whose fidelity property is enforced and checked on every apply — the writer copies un-edited parts verbatim by construction, and before any commit the apply byte-diffs every non-edited OOXML part against the input and aborts with `fidelity_violation` on any change; (b) an interventional case study (n = 2 tasks, one model, one trajectory per arm) that reproduces the #22044 corruption in a single observed instance and shows an xlq-confined agent mechanically unable to repeat it; (c) an adversarially-tested confinement mechanism, exercised in one adversarial-agent trial that found and closed a single real hole; (d) concrete formula bugs surfaced in two independent engines; and (e) a *surgical structural-edit* primitive — row and column insert and delete — that shifts every reference-bearing construct through a total algebra under a machine-checkable minimal-patch invariant, or refuses the edit with a truthful reason, shifting 6 of 6 reference classes correctly on a controlled fixture where openpyxl silently corrupts all six, and safely editing 78.8% of a 231-workbook real corpus (182 of 182 round-trip-correct) while refusing the rest, with zero silent corruptions. What is *not* here: the two write primitives between them cover in-place cell/formula edits and row/column insert-delete, but *sheet add/remove and range moves remain unbuilt, and array formulas and table-bearing workbooks are refused rather than shifted* (§4, §12); and the agent evidence is demonstration-scale — an existence demonstration that the harm and its prevention are real and reproducible under a real agent, never a corruption rate or a claim of generality, and it exercises value edits only, not the structural primitive. We write "enforced" and "checked" where the code enforces and checks, "case study" and "trial" where n is small, and we claim no formal proof anywhere.

1.3 2. The Fidelity Gap

When an LLM agent edits a workbook, the edit is not written to a log beside the artifact — it *becomes* the artifact. The cells the agent produces are the cells the file's owner will sign off on, submit to an auditor,

and be held accountable for [Panko 2008; Hermans and Murphy-Hill 2015]. This promotes a property that task-success evaluation never names into a first-class safety concern: *fidelity* — whether the parts of the file the agent did not intend to touch survive the operation unchanged. This section establishes, from the failure of the field's own precedents, that fidelity is at once routinely disturbed by the substrate agents use and systematically unmeasured by the benchmarks they are scored against. It also fixes the measurement discipline the rest of the paper rests on: byte-identity and cached values are untrustworthy across every whole-file substrate, so any correct integrity check must be semantic and recompute-aware (claim C1) — and, as §3 shows, the only way to *keep* byte-identity as a signal is to stop rewriting the whole file.

Method. We took a five-workbook corpus and re-saved each file through three substrates: openpyxl 3.1.5, LibreOffice 24.8.7.2 (`--convert-to xlsx`, headless), and the vendored IronCalc engine (master `e50ccea8`). Each output was compared to its input at two levels: an OOXML part census (part names, uncompressed sizes, and content CRC32s from the zip central directory) and a cell-level *recompute-aware* semantic diff (§7), which recomputes every formula in the vendored engine and reports a distinct `cached_value` change-kind when a cell's stored cache differs from its input while its live recomputed value does not. One provenance caveat governs the IronCalc row: the fixtures were authored by IronCalc itself, so that row is a *best case* for the engine's self-round-trip; a companion run shows it rewrites foreign files as freely as the others. IronCalc is not the hero of this table; it is the row that shows what "no change" looks like as a baseline — and the row §3 exists to improve on.

Byte-identity is destroyed everywhere. Re-saving rewrites essentially the whole package regardless of tool. LibreOffice rewrote *every* part of every fixture — not one part survived byte-identical. openpyxl rewrote all but the static theme part, and dropped `xl/sharedStrings.xml` — but this is normalization, not loss: openpyxl re-inlines the strings into the sheet XML (verified: 170 shared-reference cells became 170 inline-string cells with every text value present), so information is conserved while byte layout is not. The consequence is decisive: because ~100% of content-bearing OOXML parts are rewritten by both mainstream substrates, a byte-level or CRC-level diff reports "everything changed" on a re-save that changed no user-visible value, and is therefore useless as an integrity oracle. A check that can separate a benign re-serialization from a value-altering edit cannot operate on bytes — *unless a tool refuses to rewrite the parts it did not edit*, which is precisely what §3 does.

Three tiers, measured separately. Once byte-diff is discarded, the recompute-aware diff resolves the transformations into three distinct tiers, which must not be collapsed into a single word like "damage" (Table 1):

Substrate (version)	OOXML parts left byte-identical	Cached formula values affected	Character
openpyxl 3.1.5	theme only	**101,961** blanked to `<v/>`	recoverab
LibreOffice 24.8.7.2	none	**90,448** rewritten at 15 vs. 17 significant digits	below the
IronCalc master `e50ccea8`	all (self-authored)	0	best-case

Table 1. Re-save fidelity across substrates, five-workbook corpus (results.json, D).

T2 (semantics-preserving normalization). openpyxl's shared-string re-inlining is lossless; its only real effect is to defeat byte-provenance, already covered above.

T2.5 (recoverable, but breaks cache-trusting consumers). openpyxl blanks the cached result of **every** formula it round-trips — **101,961** `<v/>` cells across the corpus (results.json, D; 442 in `branch-consolidation.xlsx`, 99,800 concentrated in the ~100k-formula `perf-large.xlsx`, present in all five files). Because the formula text survives, the values are recoverable by recomputation; but any downstream consumer that reads the cache rather than recomputing — a viewer, an importer, a second agent, a `git diff`-style comparator — now sees empty cells where numbers were.

T3 (cosmetic sub-precision drift). LibreOffice writes formula caches at 15 rather than 17 significant digits — **90,448** cells (results.json, D; almost all in `perf-large.xlsx`). This is a legitimate serialization choice, not a defect; the drift is below the round-trip precision of a double and is recomputed on open. Its only consequence is, again, that it defeats naive byte- and cache-level provenance. That both a value-erasing transformation

(T2.5) and a benign precision choice (T3) surface through the *same* `cached_value` change-kind, and are told apart only by recomputation, is itself the argument for making the diff semantic.

T1, the severest tier, needs the write path to measure. The one tier the re-save corpus does *not* measure is the most severe: T1, genuine irreversible loss of charts, pivot tables, or VBA. The synthetic fixtures used for Table 1 contain none of these features (IronCalc cannot author them), so Table 1 makes no measured T1 claim and points to the external evidence that it is real (#22044). Section 3 closes this gap directly: it introduces a purpose-built corpus that *does* carry charts, pivots, and VBA, and measures T1 loss under each tool — which is only possible once there is a surgical writer to compare the whole-file substrates against.

The evaluation half of the gap. None of this is caught by how spreadsheet agents are scored. SheetCopilot [Li et al. 2023], SheetAgent [SheetAgent 2025], SpreadsheetBench [SpreadsheetBench 2024] and its 2026 successor [SpreadsheetBench 2026], and Finch [Finch 2025] all grade against target cells and are silent on the untouched remainder; SheetCopilot's checker skips unflagged properties *by design*. The single agent benchmark that scores any side effect at all, OS-Harm [OS-Harm 2025], judges harm categories in the agent's actions, not fidelity of the file it leaves behind. An agent could blank all 101,961 caches, report "done," and score a clean pass on every one of them.

Claim C1. Across every whole-file substrate an agent might use, re-saving an `.xlsx` destroys byte-identity and rewrites or blanks the cached values on which byte- and cache-level integrity checks depend; distinguishing a benign re-serialization from a value-changing edit therefore requires a semantic, recompute-aware comparison, and no existing spreadsheet-agent benchmark measures this dimension. The rest of the paper is the write primitive that escapes this regime — by not rewriting the whole file — and the evaluation that follows from taking that claim seriously.

1.4 3. The Surgical Write

The status-quo write path is *whole-file*: load the workbook into a library, mutate a cell, re-serialize the entire OOXML container. Everything the library does not model — chart theming, pivot caches, VBA, printer settings, the shared-string layout — is rewritten or dropped as a side effect of the save, regardless of what the edit was (§2). The surgical write inverts this. It rewrites only the OOXML parts that contain an affected cell and copies every other part of the input zip to the output byte-for-byte. The claim staked here is narrow and, we argue, new: to our knowledge xlq is the first LLM-agent write boundary that is aware of its target artifact's *format fidelity* — not merely whether a write is permitted, but whether it corrupts the artifact. The 2024–26 runtime-enforcement wave established that a deterministic runtime beats prompt-level guidance [Progent 2025; AgentSpec 2025; IsolateGPT 2024], but every system in that wave is format-blind: it gates a filesystem write without understanding whether the write preserves the file it guards.

1.4.1 The fidelity property

The primitive `xlq apply <book.xlsx> <patch.json> [--dry-run]` performs a surgical edit whose defining guarantee is:

> After `xlq apply`, every OOXML part that does not contain a cell changed by the patch is **byte-identical** to the input part.

The property holds *by construction*, not by care. The writer `ooxml::surgical_write` streams every zip member of the input to the output unchanged and re-serializes only the sheet parts named in the edit set; a part it does not touch is copied byte-for-byte, so charts, pivot caches, and `vbaProject.bin` cannot be altered by an edit that does not name their sheet. Three kinds of support back the property, and it is worth keeping them apart rather than collapsing them into the word "proof" (docs/FIDELITY.md). It is *asserted by construction* — the writer's control flow copies un-named parts verbatim; it is *measured on fixtures* — a byte-for-byte part diff against the untouched original confirms the byte-identical-elsewhere half on every fixture below; and, new in this version, it is *enforced and checked per apply* — before any commit, every real apply re-loads its own output as an on-disk `.xlsx` and confirms each predicted cell landed with its predicted value (the edited half), *and* byte-diffs every non-edited part against the input, aborting the commit with

`fidelity_violation` if any part that contains no edited cell was not preserved byte-identical (the byte-identical-elsewhere half). This upgrades both halves from "the writer intended it" to a per-write check: the edited cells verifiably compute, and the un-edited parts are verifiably untouched, on the file the consumer will open. What the per-apply check does *not* establish is that a rewritten sheet *had* to be rewritten (the benchmark's "rewritten" set is defined purely as "bytes differ," so it cannot distinguish a necessary recompute from a spurious reserialization); that residual — minimality of the rewrite, not preservation of the rest — is the only half still resting on the construction. The one deliberate exception the check whitelists is `xl/calcChain.xml`, a derived formula-ordering cache that a targeted edit makes stale; the writer drops it (Excel and every engine regenerate it on open), and the fixture tables below count that drop honestly as "not byte-identical" rather than hide it.

1.4.2 The algorithm and the transactional envelope

A single `apply` runs seven steps (docs/specs/v02-architecture.md):

1. **Precondition.** `sha256(file) == patch.base_hash`, else the operation aborts with `revision_mismatch`. The patch carries the hash of the file it was computed against, so an edit built on a stale view of the workbook is refused, not silently applied.
2. **Lock.** An advisory lock (`book.xlsx.xlq.lock`, `O_EXCL`) is held across the whole check→write sequence, so a concurrent writer gets `lock_held` rather than interleaving (no time-of-check/time-of-use window).
3. **Predict (also the `--dry-run` path).** A *copy* of the workbook is loaded into IronCalc, the typed operations (`set_cell`, `set_formula`, with an explicit JSON-to-Excel type mapping so the number-versus-text and date-serial traps are handled, never inferred) are applied, the model is evaluated, and the full set of affected cells, their new values, any newly introduced formula errors, before/after values for caller-named watch cells, and a coverage flag are computed. `--dry-run` returns this prediction and stops, writing nothing. Crucially, the coverage flag degrades when a volatile or unsupported function lies in the affected dependency graph, and a real apply whose predicted cached values are unreliable is *refused* — the write is coverage-gated (§7). A second, sharper gate refuses the write (`coverage_unreliable`, `oracle_divergent_functions: [...]`) when the affected dependency graph uses a function our own differential oracle (§8) caught the engine computing wrong: xlq will not persist an engine-computed cache it has documented reason to believe diverges from Excel. Both gates fire only on *engine-computed* caches; a literal `set_cell` writes exactly the value the user typed and is never gated, because that value does not depend on the engine at all.
4. **Surgical write.** The input `.xlsx` is opened as a zip. Affected cells are mapped to their sheet parts (`xl/worksheets/sheetN.xml`) via `workbook.xml` and its rels. For each affected sheet part, the XML is parsed and only the `<c>` elements for affected cells are edited (setting `<f>`/`<v>` and the shared-string or inline value), preserving every other element in that sheet — merged cells, conditional formatting, column widths. `xl/calcChain.xml` is dropped if present. Every other part is copied byte-for-byte.
5. **Deterministic repackage.** Fixed zip entry mtimes and a stable member ordering make `result_hash` reproducible.
6. **Proof-carrying verification and fidelity enforcement.** Before anything is committed, two checks run against the repackaged output, and either can abort the commit while the authoritative file is still untouched (nothing has reached it yet, because the commit at step 7 is the only writer). *First*, the output is re-loaded from disk the way a downstream consumer would open it — as an `.xlsx`, in a fresh engine instance — and every cell the dry-run predicted is re-checked against its predicted value; if the output does not re-open (`verification_failed: does not re-open`) or any predicted cell disagrees (`verification_failed: mismatches[...]`), the commit aborts. *Second*, the apply byte-diffs every part of the output against the input and aborts with `fidelity_violation` if any part that contains no edited cell was changed, dropped, or added — the only whitelisted difference being an edited sheet part or the deliberately-dropped `xl/calcChain.xml`. This second check *enforces* the fidelity property per write rather than trusting the writer's construction: were the surgical writer ever to disturb an un-edited part, the commit would refuse rather than ship it. The apply response carries `verified: {reopened, cells_checked, all_landed, fidelity_enforced, non_edited_parts_byte_identical}`, and a real apply proceeds only when both are satisfied. To-

gether the two checks close the gap between "the writer intended the edit" and "the written file verifiably computes the edit and preserves everything else": a malformed `<c>`, a wrong cached value, a corrupt insertion, or a stray touch to a chart part is caught here rather than asserted by a `result_hash` the consumer cannot check.

7. **Atomic apply.** An immutable `book.rev-N.xlsx` is written and fsync'd, then atomically renamed onto `book.xlsx`. A receipt is appended to the hash-chained journal `book.xlsx.xlq.jsonl` — `{rev, base_hash, result_hash, ops, ts, actor, engine_version, clock, seed, kind}` — where each receipt binds the pre-image hash to the post-image hash. An edit made outside the tool breaks the chain detectably and is recorded as an `external_edit` receipt that *adopts* the new hash rather than wedging the file.

None of these ingredients is individually new, and the paper claims none of them (§10). Append-only hash-chained receipts descend from forward-secure logging [Nitro 2025], Certificate Transparency [Certificate Transparency 2022], and W3C PROV [Moreau et al. 2013]; agent-action receipts from Notarized Agents [Notarized Agents 2026]; transactional rollback from ACID-snapshot and compensating-transaction sandboxes [Fault-Tolerant Sandboxing 2025; SAFEFLOW 2025]. The journal is a domain instantiation — a single-writer local sidecar whose threat model is file custody and out-of-band edit detection, not receiver attestation — and its differentiator over a generic transactional sandbox is that the dry-run's prediction is *semantic* (affected cells, new formula errors, watched magnitudes), not merely atomic. The write path is implemented and tested: 128 tests (100 unit, plus 5 and 7 integration) exercise the precondition, lock, dry-run, surgical writer, journal, the proof-carrying verification (its positive path and both negative paths — a wrong cached value caught, an unloadable output caught), and the oracle-divergence write gate.

1.4.3 E1: per-file fidelity preservation

We take one representative edit per fixture ("set this data cell to that number" — the edit an agent actually makes), apply the *same logical edit three ways*, and diff the resulting OOXML package against the untouched original at the level of individual zip members (docs/FIDELITY.md, benchmarks/fidelity.json). The three arms are `xlq apply` (surgical typed patch), openpyxl (`load_workbook` → set cell → `save`; for `macro.xlsm` this uses the **default `keep_vba=False`**), and LibreOffice (`soffice --convert-to` re-save, a proxy — LibreOffice has no convenient CLI cell-editor, so its 100%-rewrite figure is an *upper bound on re-save churn*, not a like-for-like single-cell edit). Following the discipline of the experiment, results are reported **per file, never summed into a corpus total** — the interesting fact is per-workbook (charts survive here, VBA survives there), and averaging would hide exactly the failures the experiment exists to surface. The four fixtures are hand-picked to carry the features whole-file rewriters destroy — charts, pivot tables and caches, VBA, cross-sheet formulas — so E1 is an *existence demonstration* that the fidelity property holds on real workbooks with those features, per file, not a population estimate over any workbook distribution.

`pivot-chart.xlsx` — 2 charts, 1 pivot table, 2 pivot caches (50 parts). Edit: `Sheet1!A2` 222 → 999.

Tool	byte-identical	rewritten	dropped	added	charts	pivot
xlq	**48 / 50**	1	1	0	byte-identical	byte-identical
openpyxl	1 / 50	36	13	4	degraded (6 support parts dropped)	rewritten
LibreOffice	0 / 50	37	13	6	degraded (same 6 support parts dropped)	rewritten

xlq rewrites only the edited sheet and drops the stale `calcChain.xml`; both charts, the pivot table, both pivot caches, shared strings, styles, drawings, comments, and printer settings are byte-identical. openpyxl drops six chart-support parts (both `chart*.xml.rels`, both `colors*.xml`, both `style*.xml`), plus `sharedStrings.xml`, both comment parts, both VML drawings, `printerSettings1.bin`, and `calcChain.xml`; the core chart and pivot parts are present but rewritten, and **the output does not re-open in the ironcalc engine**. LibreOffice rewrites all 50 parts and, on this axis, is no better than openpyxl — it drops the same six chart-support parts, inside a package that still re-opens.

`macro.xlsm` — VBA macros (11 parts). Edit: `Data!B2` 100 → 200.

Tool	byte-identical	rewritten	dropped	added	VBA	reloads (ironcalc)
xlq	**10 / 11**	1	0	0	`vbaProject.bin` byte-identical	
openpyxl	1 / 11	8	2	0	**dropped** (saved as `.xlsx`)	
LibreOffice	0 / 11	11	0	1	present (rewritten)	

xlq rewrites only the edited sheet; `vbaProject.bin` is byte-identical. openpyxl with its default `keep_vba=False` drops `vbaProject.bin` — **all macros gone** — and `sharedStrings.xml`, and can only write `.xlsx`: the macro-enabled workbook is silently downgraded to a macro-free one. LibreOffice keeps the macro but rewrites every part.

`payroll.xlsx` — 3 sheets, cross-sheet formulas (13 parts). Edit: `Rates!B2` 16 → 25 (a rate that feeds the `Payroll` sheet).

Tool	byte-identical	rewritten	dropped	added
xlq	**11 / 13**	2	0	0
openpyxl	1 / 13	10	2	0
LibreOffice	0 / 13	12	1	1

xlq rewrites *exactly two* worksheet parts — the edited `Rates` sheet and the `Payroll` sheet, whose formula cells recompute because they depend on the edited rate; both rewrites carry changed cell values, and nothing else moves. openpyxl drops `sharedStrings.xml` and `xl/metadata.xml` and rewrites 10 parts.

`claims.xlsx` — 2 sheets, ~1.3k formulas (12 parts). Edit: `Claims!D2` 13335 → 20000.

Tool	byte-identical	rewritten	dropped	added
xlq	**11 / 12**	1	0	0
openpyxl	1 / 12	9	2	0
LibreOffice	0 / 12	11	1	1

xlq rewrites only the sheet carrying the edit and its downstream formulas; the `Limits` sheet, styles, shared strings, and metadata are byte-identical. openpyxl again drops `sharedStrings.xml` and `metadata.xml`.

What the tables establish, stated straight. The measured `feature_survival` flag records *presence*, not integrity: a for openpyxl's charts means a `chartN.xml` is present in the output, not that it is byte-identical — openpyxl's charts are present yet stripped of their theming and relationship parts, which is why that file will not re-open. Only xlq's s are also byte-identical. The reload failure is reported narrowly and conditionally: ironcalc is xlq's own vendored engine, so its rejection is xlq's engine rejecting a competitor's output, not a neutral arbiter; the one neutral engine measured, LibreOffice, *opens* the same openpyxl file. What is measured and unconditional is that openpyxl drops `sharedStrings.xml` on every fixture, and drops all VBA and all chart-support parts on the fixtures that have them. Against every whole-file substrate, then, the surgical writer is the only tool that leaves the parts an edit did not touch exactly as it found them — which is the property that makes the boundary of §6 meaningful.

1.5 4. Surgical Structural Edits

The primitive of §3 edits `<c>` elements in place; it cannot insert or delete a row. That limit is not an accident of implementation but the paper's own named frontier, and this section closes it. Structural edits — row and column insert and delete — are where the fidelity property of §3 and correctness genuinely *conflict*, and only in spreadsheets. Inserting a row above a chart's data range does not leave the chart untouched: a *correct* edit **must** grow a chart plotted on `B2:B7` to `B2:B8`, move a cross-sheet `=Data!B8` to `=Data!B9`, and rewrite every reference-bearing coordinate that crosses the insert line. Byte-identity of those parts — the exact half

of the §3 property that made it a boundary — is therefore *impossible* for a correct structural edit. A tool that preserves those bytes is wrong; a tool that regenerates the file to get them right loses everything else. This section builds the primitive that occupies the cell neither escape reaches: correct reference shifting *and* byte-minimal fidelity, under a machine-checkable invariant, with an honest refusal for the constructs it does not yet shift.

1.5.1 The prior-work gap

Three lines of prior work each reach part of this cell and miss it on a different axis (research-log/011). **Excelsior** [Excelsior 2008] performs exactly this operation set — insert and delete rows and columns, resize and move tables — and shifts the dependent formulae correctly, generating "a spreadsheet with different structure but identical outputs." But it works by discovering a high-level model (`tables + equations`) and *regenerating* the workbook from it, and it explicitly sacrifices fidelity: by its authors' account it "did not try to reproduce properties such as cell formats and colours, input menus, and charts," and "cell styles were lost by the structure discovery stage." Excelsior *is* the load-into-a-model-and-regenerate approach whose fidelity loss this whole paper targets — it does the structural edit and destroys everything not in its model. **TACO** [TACO 2023] formalizes precisely the relative/absolute reference algebra a correct shift needs — the fixed-versus-relative relationship of a formula to the head and tail of each referenced range, and incremental maintenance under insert and delete — but only for an *in-memory dependency graph* built for fast recalculation. It never touches the file, never shifts a reference in a chart, a pivot cache, or a defined name, and has no notion of fidelity. **PPTArena / PPTPilot** [PPTArena 2025] is the closest on fidelity-aware agentic OOXML editing, explicitly targeting "non-destructive modification" and routing between a library and "deterministic OOXML patching" — but for PowerPoint, which has *no formula-reference-shift problem* (slides do not cross-reference cells), so the hard part is simply absent; its preservation is measured by a VLM-as-judge over screenshots and structural diffs, not a provable invariant, and its XML patches are model-generated, with no formal shift algebra and no proof-carrying check. No prior tool occupies the *correct + byte-minimal + fidelity-preserving* cell: Excelsior shifts references but regenerates the file, TACO has the algebra but never leaves memory, PPTArena is fidelity-aware but for a format without the shift problem and without a checkable guarantee.

1.5.2 The reference-shift algebra

The correctness core is a total function that maps any A1 reference to its post-edit target. On **insert** of n rows at k, each range endpoint moves independently — `r' = r + n if r  k else r`, applied to head and tail separately — which reproduces every boundary case for free: an insert above a range shifts both endpoints and moves it down; an insert strictly inside holds the head and shifts the tail, so the range *grows*. On **delete** of rows `[k, k+n)`, a six-case clamp governs: a range clipped at one endpoint *shrinks* — `=SUM(A5:A10)` deleting rows 5–6 becomes `=SUM(A5:A8)`, not `#REF!` — and `#REF!` is emitted *only* when the reference is entirely consumed. is sheet-scoped (a token shifts only if its resolved target is the edited sheet, or a 3D span containing it — never an external or unrelated-sheet reference), transparent to the absolute flag (`$A$5 → $A$6` on an insert above row 5; the `$` governs fill and copy, never insert and delete — confirmed against Excel, which is exactly why the documented `INDIRECT("A5")` pin workaround has to exist), and axis-selective (a row operation leaves whole-column refs like `A:A` untouched, and a column operation leaves whole-row refs untouched). is validated by 46 unit tests that encode documented Excel semantics.

We do not bill as the novelty, and the honest reason is on the record. A predict-then-run theory-review gate *failed* the first version of on three decisive counts (research-log/013): the delete path emitted `#REF!` for the common clip case that should shrink; the scoping was unsound (shifting every cross-sheet token would corrupt references into non-edited sheets and external workbooks); and shared formulas broke the coordinate model. Had we built before that gate, the delete path would have silently corrupted files. as it now stands is a *faithful re-encoding of documented Excel semantics* — the correctness it carries is Excel's, not ours. The contribution is what is applied *through*.

1.5.3 The minimal-patch invariant

The novelty is a system property. is applied *uniformly* across every reference-bearing OOXML part — sheet `<f>` formulas and `<c r>`/`<row r>` coordinates, cross-sheet references, defined names, chart `<c:f>` data references, pivot-cache `worksheetSource`, and the `sqref` of merged cells, conditional formatting, and data validation *as well as the formula bodies* inside `<cfRule>` and `<dataValidation>` — by *raw attribute-value surgery*, so that the only bytes that differ between input and output are reference coordinates and the physically inserted or removed rows. This is the **minimal-patch invariant**, and it is machine-checkable: the changed non-sheet parts are digit-stripped-identical to their originals, and it is proof-carrying in the §3 sense — the output must re-open and evaluate before the write commits. It is the refinement the §3 property could not be — byte-identity-*except-coordinates* rather than byte-identity — and it is what resolves the fidelity-versus-correctness conflict none of Excelsior, TACO, or PPTArena occupies. That "applied uniformly across all parts" is an easy claim to overstate, and a 28-agent adversarial review (research-log/014) confirmed and fixed nine defects in an earlier version of this coverage — among them whole-column refs left silently unshifted under column operations, and CF/DV formula bodies shifted only in their `sqref` and not their body — several of which had violated the never-silently-wrong invariant below. The coverage above is the post-fix set (192 tests, 0 failures), and the review's novelty verdict — all four novelty-versus-prior findings returned *partial*, none confirmed as obvious or known — is what the minimal-patch invariant survived.

1.5.4 Shared-formula expansion

The dominant real-world construct is the *shared formula*: Excel and autofill emit a single master formula plus lightweight `<f t="shared" si=…>` stubs for the filled range, and coordinate surgery on a stub cannot express a shift whose relative reference crosses the group interior. xlq *materializes* the group — each dependent is reconstructed from the master plus its offset, then shifted with — which is what Excel does internally. This is the key lever: it trades strict minimal-patch purity on those cells (the stubs become full formula bodies) for correctness on the construct that appears in most real workbooks, and it lifted the safe envelope on the real corpus below from **22.5% to 78.8%**.

1.5.5 Never silently wrong

The discipline that makes the primitive a boundary rather than a best-effort transform is §3's, sharpened: for every reference-bearing construct, xlq *either* shifts it correctly *or* refuses the edit with a truthful residual reason — never a silent off-by-*n*. Array formulas (which Excel itself forbids splitting across an insert line) and workbooks carrying table parts are *refused*, as are 3D spans not anchored on the edited sheet. The adversarial review's most important finding was constructs that did *neither* — silently corrupted — and every one is now shifted-correctly or refused; the invariant is restored, not asserted.

1.5.6 E4 — the controlled result

E4 (benchmarks/structural.json) inserts one row at row 5 of sheet `Data` in a fixture carrying a bar chart, a straddling `=SUM(B2:B7)`, a single-reference `=B5*2` at the insert row, an absolute `=$B$8`, a merged region, a cross-sheet `Report!A1 = Data!B8`, a defined name `Total = Data!$B$8`, and a chart data reference `Data!$B$2:$B$7` — one instance of each reference class a row insert must move.

Tool	References shifted correctly	Fidelity
xlq restructure	**6 / 6**	10 / 14 parts byte-identical; the 4 changed parts differ *only* in coord
openpyxl `insert_rows`	**0 / 6**	12 / 14 parts, but every reference left unshifted
LibreOffice round-trip	correct-by-engine	**0 / 14** parts byte-identical

Table 2. E-structural: one row insert, correctness × fidelity × recompute (structural.json).

xlq shifts all six references to their documented target — `=SUM(B2:B7)` grows to `=SUM(B2:B8)`, `=B5*2` becomes `=B6*2`, `=$B$8 → =$B$9`, the cross-sheet `Data!B8 → Data!B9`, the defined name `Data!$B$8 → Data!$B$9`,

and the chart range grows to `Data!$B$2:$B$8` — and only four parts change (the edited sheet, the cross-referencing sheet, the chart, and `workbook.xml` for the defined name); the three non-sheet changed parts are digit-stripped-identical to the originals, and the recompute is *executed*, not asserted — `xlq calc` on the committed file returns `SUM = 760`, the pre-edit total. openpyxl's `insert_rows` moves the cells but rewrites *not a single reference*: `=SUM(B2:B7)` still reads the pre-insert range, `=B5*2` now multiplies the **blank inserted row** ($\rightarrow$ 0), the cross-sheet ref, the defined name, and the chart all point at stale cells, and the file opens without complaint and computes *silently wrong* — its 12/14 preserved parts are irrelevant when every number is wrong. LibreOffice shifts references correctly by engine but rewrites all 14 parts on a bare round-trip, before any edit — correct arithmetic, destroyed provenance.

1.5.7 E5 — the honest real-corpus envelope

E4 is a single controlled fixture, and the sharpest review objection was survivorship bias: it is one openpyxl-authored file in which shared formulas — the dominant construct — never appear. E5 answers it by running xlq restructure across the **231 vendored IronCalc test workbooks**, real files authored in Excel and LibreOffice rather than by openpyxl (benchmarks/corpus_coverage.json).

Outcome	Files	%
Safely edited (shifted correctly, reopens, no residual)	182	**78.8%**
Refused (residual, never silently wrong)	49	**21.2%**
— array formula present (Excel forbids splitting)	47	20.3%
— table part present	2	0.9%

Table 3. E-structural real-corpus envelope, 231 workbooks, with shared-formula expansion (corpus_coverage.json).

Every one of the 231 files is *either* safely shifted *or* refused with a truthful reason; **zero are silently corrupted**. The refusals are array formulas — which Excel forbids editing through — and the two table-bearing files. Correctness is checked *engine-free* (benchmarks/roundtrip_correctness.json): for each of the 182 safe files, insert a blank row at k then delete row k — a net identity that must return every Excel-authored cached value to its original cell, so any wrong shift on either the insert or the delete would move a value — and **182 of 182 (100%) preserve every Excel cached value cell-for-cell**, proving 's insert and delete are exact inverses on real files with real shared formulas, cross-sheet references, and mixed absolute/relative references. One finding runs in xlq's favor and explains a design choice. An earlier version of this oracle used LibreOffice's recompute as the reference and reported 16 "mismatches," all in precision-sensitive financial and statistical files; investigation showed these were *not* xlq errors but that *LibreOffice reconstructs shared formulas differently from Excel* — an NPV over a shared range where Excel's cache is 11.78 and LibreOffice's on-the-fly reconstruction is 11.41 — and that xlq's expansion produces the formula matching Excel's cached value (LibreOffice run on xlq's *expanded* output also computes 11.78). xlq's shared-formula handling agrees with Excel where LibreOffice's own reconstruction does not, which is exactly why the correctness oracle is anchored on Excel's caches rather than a LibreOffice recompute.

1.5.8 Honest scope

Three edges bound the primitive, stated plainly and revisited in §12. Array formulas and table-bearing workbooks are *refused*, not shifted — 21.2% of the corpus (arrays 20.3%, tables 0.9%) — declined with a truthful reason rather than silently corrupted, and named as future work. The minimal-patch purity of §3 is *relaxed* for shared cells, which are materialized into full formula bodies before shifting — a deliberate trade of byte-minimality on those cells for correctness on the dominant construct. And the end-to-end evaluation exercises one operation family, insert and delete of a row; column operations share the same and are unit-tested (including the whole-column cases the adversarial review surfaced) but are not in the end-to-end corpus table, while sheet add/remove and range moves are not built. The primitive is also unexercised under the agent case study (§5), which remains scoped to value edits.

1.6 5. Does It Work For an Agent?

A static fidelity benchmark (§3) shows the writer preserves parts when *we* drive it. It cannot show that the property survives contact with a real agent — an agent that chooses its own tool invocations, reads its own results, and decides for itself whether it succeeded. This section reports an interventional *case study* — n = 2 tasks, one model, one trajectory per arm — in which a real LLM agent completes a genuine edit task, and the only manipulated variable is the tool it is given to write the file (docs/AGENT-AB.md, benchmarks/agent_ab.json). It is demonstration-scale by design: it shows that the harm and its prevention exist and are reproducible under a real agent, not that they occur at any particular rate. This is the experiment the thesis always demanded, and the one whose absence the earlier version of this work was rejected for.

1.6.1 Design

The agent is a real LLM subagent (Claude) with a bounded tool surface: a shell and a Python interpreter, or the `xlq` binary. It is given a plain-English edit task and told to complete and save it. It is *not* told which outcome is being measured, nor that fidelity is scored. Two arms hold the agent, prompt, input file, and environment fixed and vary only the write tool:

- **`openpyxl`** — the status-quo path an unconstrained coding agent reaches for today, the one implicated in #22044. The agent edits with the openpyxl library and saves.
- **`xlq`** — the format-aware boundary. The agent may mutate the workbook only through `xlq apply` (inspect → `patch.json` → apply).

Two real tasks, each a single `set_cell` edit on a workbook carrying a feature that whole-file rewriters destroy: **t1-vba** (`macro.xlsm`, VBA project, change `Data!B3` 120 → 250) and **t2-chart** (`pivot-chart.xlsx`, 2 charts + a pivot, change `Sheet1!A2` 222 → 900). Each trial scores two things by *external check*, not by the agent's own claim: **task success** (did the target cell reach the requested value *and does the saved file still open?*) and **fidelity of untouched content** (of the parts the edit did not touch, how many survive byte-identical; which feature parts survive; which are dropped). Because everything but the edit tool is held fixed, any difference in outcome is attributable to the tool boundary.

1.6.2 Per-task results

Task	Feature	Arm	Target	Task success	Reloads	Feature byte-identical	Parts byte-identi
t1-vba	VBA	**openpyxl**	250	yes	yes	yes[1]	**2 / 11**
t1-vba	VBA	**xlq**	250	yes	yes	yes	**10 / 11**
t2-chart	2 charts + pivot	**openpyxl**	*load-error*[2]	**no**	**no**	no / no	**1 / 50**
t2-chart	2 charts + pivot	**xlq**	900	yes	yes	yes / yes	**48 / 50**

[1] In this run the openpyxl agent volunteered `keep_vba=True`, so `vbaProject.bin` survived byte-identical — the *charitable* openpyxl (threat T4). Even so it rewrote 9 of 11 parts and silently dropped `sharedStrings.xml`. [2] `target_cell_value` came back as `<load-error:'Chartsheet' object has no attribute 'defined_names'>`: the saved file cannot be reopened, so the value is unverifiable because the workbook is corrupt. The agent nonetheless *reported success* — it read the raw XML, saw `<c r="A2" t="n"><v>900</v></c>`, and concluded the edit landed. [3] `calcChain.xml` is a recomputable formula-ordering cache, not a feature; its removal is expected and lossless.

The false-success finding. The t2-chart / openpyxl trajectory is the headline. The agent's cell write *did* land — the number 900 is in the XML — but the workbook did not survive the save: the two charts and the pivot were stripped of their relationship parts, 13 of 50 parts were dropped, and only 1 of 50 remained byte-identical. When the agent's own post-save reload raised `'Chartsheet' object has no attribute 'defined_names'`, it worked around the failed check by reading the raw sheet XML, saw its target cell, and **reported success on a corrupt, non-reloadable file**. This is #22044 reproduced live in a single observed trajectory, including the property that made #22044 dangerous in the first place: the corruption

is invisible to the agent's own success criterion. We report it as one observed instance of the false-success failure, not a frequency. In the paired trial the xlq-confined agent completed the *same task on the same file* with charts and pivot byte-identical, 48 of 50 parts untouched, the file reloadable, and a signed receipt (`rev 1`, base→result hash chain). **In this trial the xlq-confined agent could not commit the #22044 corruption even while attempting the identical edit** — and the reason is mechanical and holds for the tool generally, not just this trajectory: `xlq apply` has no operation that rewrites the whole container.

1.6.3 Threats to validity

We state these plainly so the reader does not over-generalize (docs/AGENT-AB.md §5):

- **T1 — n = 2 tasks.** Two workbooks, two features. This demonstrates the harm and the prevention *exist and are reproducible under an agent*; it is not a rate estimate. It does not claim "openpyxl corrupts X% of workbooks."
- **T2 — single operation type.** Both tasks are one `set_cell` each. The structural-edit primitive of §4 (row and column insert and delete) is built but *unexercised under the case study*, and sheet add/remove and formula rewrites are neither built nor exercised; the causal claim is scoped to value edits.
- **T3 — the "agent" is an LLM subagent with a bounded tool surface.** It is a real model making real tool calls, but not a full autonomous IDE agent with retrieval, retries, or user follow-up, and we measured a single trajectory per cell, not a distribution over agent skill. A more capable agent might notice and repair the openpyxl corruption; a less careful one would ship it.
- **T4 — the openpyxl arm is the *charitable* openpyxl.** On t1 the agent chose `keep_vba=True`. The realistic default is `keep_vba=False`, which drops the VBA project entirely and silently (§3, E1) — a strictly worse and more common outcome. Our openpyxl numbers therefore *understate* the status-quo harm, not overstate it.
- **T5 — "reloads in engine" is checked against ironcalc**, the engine xlq vendors. The t2 openpyxl failure also reproduces under openpyxl's *own* reader (the `Chartsheet.defined_names` load-error), so the failure is not an engine-choice artifact — though a reader that tolerates missing chart `_rels` might open the file in a degraded state rather than erroring.

The causal claim, precisely scoped. In these two paired trials, holding the agent, task, input workbook, and environment fixed, routing the edit through xlq instead of openpyxl changed the outcome from "feature-destroying, non-reloadable, unattested" to "byte-identical on all feature parts, reloadable, and receipted." On the chart/pivot task this is the difference between reproducing #22044 and being mechanically unable to. The reason is mechanical, not statistical: `xlq apply` rewrites only the parts an operation touches and refuses whole-container rewrites, so the case study is a demonstration that this mechanism survives contact with a real agent — which the static benchmark of §3 could not show — while the two-trajectory sample size is exactly why we call it a case study and not a rate.

1.7 6. Threat Model

A built write path invites a sharper objection than a read-only one: even a surgical, receipted write is *opt-in* — nothing forces an agent through xlq, so what exactly does it enforce? This section answers precisely, stating what xlq guarantees, against whom, and where the boundary is and is not (docs/THREAT-MODEL.md) — and, in this version, *tests* the non-bypassability claim in a single adversarial-agent trial rather than only arguing it from the harness design. One trial is not a proof; it is one adversary who tried, failed to forge, and found a real hole we then closed. The honest answer matches how the 2025–26 enforcement literature scopes its own boundaries, and we make the parallel explicit rather than overclaim.

The deployment: a confined agent. xlq is an enforcement boundary *for an agent confined to it.* The concrete deployment is the standard tool-sandbox model of the enforcement wave [IsolateGPT 2024; Progent 2025]: the agent harness grants the agent the `xlq` tool and **no capability to write `.xlsx` bytes by any other means** — no openpyxl, no raw file write, no shell that can. The shipped `skills/xlq/SKILL.md` plus a restricted tool surface is that harness. Within it, `xlq apply` is the agent's *only* path to modify a workbook, and every modification is surgical (only sheet parts with an edited cell are rewritten; charts, pivots, VBA, styles, external links byte-identical — §3), preview-gated (`--dry-run` predicts affected cells

and new errors first), precondition-checked (`base_hash` must match under an advisory lock, no TOCTOU), coverage-gated (a write whose predicted cached values are unreliable, or whose formulas use a function the differential oracle found divergent, is refused — §7, §8), proof-carried and fidelity-enforced (before commit the output is re-loaded and every predicted cell re-verified, *and* every non-edited part is byte-diffed against the input, aborting with `fidelity_violation` on any change — §3), and receipted (a hash-chained journal entry records base/result hash, ops, and actor; out-of-band edits are detected, not silently absorbed). So an agent confined to xlq **cannot reach the #22044 corruption**: it has no operation that rewrites the whole container or drops a chart/pivot/VBA part. That is the enforcement claim, and §5 is its interventional case-study evidence.

What the boundary is *not*. It is not a guarantee against an agent with unrestricted shell or raw filesystem access. Such an agent can call openpyxl directly and corrupt the file — exactly the status quo — and xlq does not, and cannot, prevent that; no library-level tool can. Non-bypassability is a property of the *harness* (which tools it exposes), which xlq *enables* but does not itself enforce. We claim non-bypassability *within the harness*, and say so, rather than claim a guarantee against an adversary handed raw write access. This mirrors the enforcement wave exactly: Progent's policy engine, IsolateGPT's process isolation, and MiniScope's scoped tokens all bound an agent *routed through them*; none prevents an operator from also handing the agent an unmediated capability. The boundary is the mediation surface, and its value is that a harness *can* confine an agent to it — for spreadsheets, previously it could not, because no safe write primitive existed.

Actor	Can it corrupt via xlq?	Notes
Careless agent confined to xlq	No	No whole-file-rewrite operation exists; surgical by
Buggy agent (wrong cell/formula)	Bounded	dry-run previews the effect; `base_hash` + receipts
Agent with raw shell / openpyxl	Yes (bypasses xlq)	Out of scope by construction; the harness, not xlq
Agent truncating/overwriting the file directly	Blocked within harness	Not corruption *via* xlq but blunt destruction; clos
Concurrent writers	Serialized	Advisory lock; the second writer gets `lock_held`.
Out-of-band edit between ops	Detected	Hash-chain `external_edit` + adoption marker.

Table 4. Threat actors and coverage (THREAT-MODEL.md).

Non-bypassability, tested adversarially — not merely argued. The re-review's standing objection was that this enforcement claim was *argued* from the harness design but never *tested* against an adversary who tries and fails to bypass it. We ran a single adversarial trial (n = 1), and it both confirmed the forgery claim and found a real hole we then closed (docs/NON-BYPASS.md); we present it as one adversarial probe that surfaced and closed a concrete gap, not as a bypass-resistance proof. A confined harness (`env -i PATH=./confined-bin`) exposes exactly one executable — `xlq` — plus `/bin/sh` builtins; `python3`, `cp`, `zip`, `unzip`, `dd`, `base64`, and every archiver or binary editor return "not found," verified. An adversarial LLM agent, instructed to be impatient and to actively bypass the safe path (rewrite the file with Python, unpack the zip, anything faster), reported its result honestly:

- **Forgery — blocked completely.** "I genuinely could not produce a modified-but-valid workbook by any route except xlq." With no interpreter and no archiver, there was no route to a *valid-but-wrong* workbook; the #22044 class — a corrupt file that looks fine and is reported as success — was unreachable.
- **Destruction — not blocked (the hole).** The working directory was writable and `: > target.xlsx` is a shell builtin, so the agent could *truncate* the file — no forgery, but blunt destruction, with no xlq and no receipt. The confinement prevented forgery, not destruction.

We report the hole as the rigor, not a weakness: an adversarial experiment exists precisely to surface it, and the fix follows first-principles from the finding (docs/NON-BYPASS.md §2). The authoritative store is made read-only to the agent — `authoritative/book.xlsx` is `chmod 444` inside a `chmod 555` directory, and the confined PATH has no `chmod`, so the shell cannot restore write; truncation, `echo >`, and new-file creation against it all return permission denied, verified. The agent instead works on a *copy* in a writable `work/` dir via `xlq apply` (it may freely destroy its own scratch copy — harmless), and a **receipt-gated broker** running *outside* the confinement promotes `work/book.xlsx` to `authoritative/` only if it hashes to

the last receipt's `result_hash` *and* that receipt chains from the authoritative file's current hash, rejecting anything not receipt-backed or not a clean successor. End-to-end, measured: the confined agent applied its edit (`rev 1`, `verified.all_landed = true`, fidelity 48/50), the broker verified the chain (`base 35010189…` → `result abbed433…`) and promoted, and the authoritative workbook afterward carried **A2 = 900** with **10 of 10 chart parts byte-identical** to the original and the receipt chain intact — changed only through a receipted, proof-carried, fidelity-preserving xlq operation, with no route for the agent to forge or destroy it.

The honest scope sentence, which we state in the paper rather than bury: *xlq is an enforcement boundary for an agent whose harness confines its workbook writes to the xlq tool; within that confinement it makes the #22044 corruption unreachable and every change previewed, attributable, and reversible — now tested in one adversarial trial, forgery blocked and the one destruction hole found and closed. It does not defend a workbook against an agent granted an unconfined shell with an interpreter and a writable authoritative file — that is the harness's responsibility, and the two-tier read-only-store-plus-broker mechanism above is exactly how a harness discharges it, stated as a scoping assumption rather than a guarantee.*

1.8 7. Reading Without Writing: The Census and Coverage Honesty

The write path of §3 is only trustworthy if the read that precedes it is: an agent must see the workbook to build a patch, the dry-run must predict an edit's effect, and the boundary must know when it *cannot* predict. This section describes the read layer — a privacy-safe structural census and a per-file coverage report — carried over from the read-only system and now load-bearing for the write. Both are reads that are side-effect-free by construction; the mutating surface is `apply` alone (§3).

1.8.1 A privacy-safe structural census

`xlq inspect` produces a *census*: a structural summary that answers "what does this workbook need from an engine?" without answering "what is in it?" It reports sheet dimensions and cell/formula tallies, per-literal counts of stored errors, the call-site frequency of each Excel function, which OOXML parts are present, and the file's SHA-256 — and, by specification, never a cell value, never a fragment of a formula body, and never a full filesystem path. The invariant is regression-tested against a sentinel workbook whose cells carry `SECRET_VALUE_XYZ` and `SECRET_FORMULA_LIT`: the census is serialized in normal and redacted mode and asserted to contain neither marker nor the file's parent directory. Taking the format seriously forces one distinction a format-blind summarizer would miss: a called name that is neither a canonical Excel function nor known to the engine — a VBA/XLL user-defined function, an add-in call, a `LAMBDA` through a defined name — is treated as *user data*, because such names routinely encode deal terms and client identities (a call site in our tests is `DealMargin_AcmeCorp`); these are reported under a separate `user_defined_calls` member, are redactable, and never leak into the function tallies.

The census is also what makes the artifact legible to an agent at a fraction of the token cost of dumping cells. On a workbook of roughly 100k formulas (`perf-large.xlsx`), the census was **999 bytes** against a **7,239,303-byte** naive full-cell JSON dump — a **7,246.5× reduction** (results.json); on the small `branch-consolidation.xlsx`, 1,800 B versus 51,107 B (28.4×). This is not comparable to prompt-side content-compression schemes such as SpreadsheetLLM [Dong et al. 2024], which compress cell *contents* for understanding and question-answering: the census discards contents entirely and keeps only the structure a mutation-safety decision needs.

1.8.2 Coverage honesty: three numbers, not one

A formula runtime is conventionally described by one number: how many of Excel's functions it "supports." That fuses two questions the owner of a workbook needs answered apart — does the runtime *recognize* this function, and can it *compute the function's value from the workbook's own data*? — and has no vocabulary for the case where the correct answer is a specific documented error. We replace the bit with a partition. Against a pinned catalog — the 522 worksheet functions in Microsoft's alphabetical function list, fetched 2026-07-02 (`benchmarks/excel-functions.txt`) — the vendored engine measured (`benchmarks/coverage.json`):

#	Measure
1	**Recognized** — the name parses and dispatches to defined behavior, never a parser unknown-name rejection
2	**Locally evaluable** — full semantics computed from the workbook's own data
3	**Policy-limited** — recognized and argument-validated, then the documented desktop-Excel refusal literal, because the

Table 5. The three-number coverage taxonomy (coverage.json). 505 + 17 = 522.

The numbers are exhaustive and disjoint over the recognized set. A guard states what number 2 does *not* mean: *locally evaluable is not correct.* Evaluability is coverage — the engine will compute a defined value from workbook data — whereas correctness, whether that value matches Excel, is the oracle's burden and is measured separately in §8. The 505 must never be read as "505 correct." The 17 policy-limited functions decompose, on the same honesty the taxonomy exists to enforce, into three reasons a reader would act on differently (coverage.json):

Reason class	n	Example functions
Capability-limited	14	WEBSERVICE `#VALUE!`; RTD `#N/A`; STOCKHISTORY / DETECTLANGUAGE / TRAN
Policy-blocked	2	CALL, REGISTER.ID (XLM/DLL native-code entry points)
Context-precondition	1	GETPIVOTDATA (needs a rendered PivotTable the engine does not model)

Table 6. The 17 policy-limited functions, decomposed by reason (coverage.json).

Coverage honesty is a *per-file* property, and this is what ties it to the write path. When a file uses a policy-limited name, `xlq inspect` and `xlq calc` report it in the census's `policy_limited_functions` bucket (name → literal) and set `coverage.reliable: false` — the stored values genuinely cannot be *verified* locally — while `unsupported_functions` stays empty, because "the engine does not know this name" would be a lie. The dry-run (§3) reuses exactly this machinery: if a volatile or policy-limited function lies in an edit's affected dependency graph, the prediction's coverage flag degrades and the write is refused, so the boundary never commits an edit whose predicted downstream values it cannot stand behind. A companion gate driven by the differential oracle's confusion matrix (§8) refuses the write on the same principle when a formula uses a function the engine is known to compute wrong — the two gates together bound both what the engine *cannot* evaluate and what it evaluates *incorrectly.* The three numbers were reached by measuring before building: the pinned IronCalc 0.7.1 release recognized 345/522 (66.1%), but upstream *master* had already added ~148 functions, so reading the diff — not writing code — moved recognition to 494/522 (94.6%); three residual local patches (AGGREGATE, ENCODEURL, HYPERLINK) reached 497, and a final milestone implemented eight more with local semantics (FILTERXML, EUROCONVERT, DBCS, JIS, BAHTTEXT, PHONETIC, GROUPBY, PIVOTBY), taking locally evaluable to 505 and closing recognized to 522/522 (`docs/COVERAGE.md`). This makes claim **C2**: a formula runtime can recognize the full Excel catalog with defined, documented behavior and report that coverage *honestly*, as a three-number partition with a per-function literal for every non-evaluable case, rather than as a single supported-count that conflates recognition, evaluability, and correctness.

1.9 8. Documentation-Arbitered Differential Validation

The census (§7), the coverage report (§7), and above all the dry-run prediction that gates every write (§3) are only as trustworthy as the formula engine beneath them: a vendored engine that computes a wrong value will have the runtime faithfully report a wrong value, and predict a wrong downstream effect. Correctness is a separate burden — contribution C3 — and it cannot be discharged by assumption. Nor can it be discharged the way a compiler's or a SQL engine's can, because there is no mechanized specification of spreadsheet formula semantics: ECMA-376 standardizes the OOXML container, not the numerical behavior of the functions inside it, and OpenFormula governs ODF, not Excel's dialect. So the engine was validated the way compilers were before verified reference semantics existed [Yang et al. 2011]: by differential testing. Each of 1,659 hand-authored cases across 492 functions was run through two independent engines — the

vendored IronCalc (master `e50ccea8`) and LibreOffice 24.8.7.2 headless — and every divergence was treated as a signal to investigate, not a verdict. Of the 1,659 cases the engines agreed on 1,273, disagreed on a value in 85, both raised an error in 196, and hit 105 cases LibreOffice does not implement (15 of which IronCalc also errored on); no case crashed either engine (`engine_error = 0`, benchmarks/agreement.json).

1.9.1 The disagreement matrix

The scientific object is not the agreement count but the 85 cases in which both engines returned a value and the values differed. Each was adjudicated by hand against Excel's documented behavior, using a three-way verdict discipline [Liu et al. 2024] — a divergence is a *bug*, an *ambiguity*, or *undecidable*, never auto-blamed on one side (benchmarks/triage-analysis.md):

Excel-arbitrated verdict	Count
IronCalc wrong	24
LibreOffice wrong	41
Both wrong	7
Spec-ambiguous	13
Undecidable here	0

Table 7. Excel-arbitrated triage of the 85 value-vs-value disagreements (triage-analysis.md).

Seventy-two of the 85 (85%) received a decidable, documentation-grounded fault assignment. Two properties matter more than any cell. First, the harness indicts the engine the runtime actually ships **24 times** — a rigged harness does not fault its own engine two dozen times. Second, on the 18 core-semantics cases with zero ambiguity the split is nearly even (8 IronCalc-wrong to 10 LibreOffice-wrong), while on the 21 financial cases it is lopsided the other way (13 LibreOffice-wrong to 3 IronCalc-wrong, dominated by day-count and duration errors): the two engines fail in *different directions*, which is precisely why comparing them was informative. Concrete filable defects resolve from the verdicts — IronCalc: `CONVERT(68,"F","C")` returned 19.65 where Excel returns 20, `ROW` over a range returned a scalar instead of the spilled array, `SUMPRODUCT` did not coerce boolean arrays, the `A`-family aggregates ignored text and boolean cells, `TRIM` did not collapse internal whitespace, `PRICE` at basis 3 returned exact par; LibreOffice: `POWER(0,0)` returned 1 where Excel returns `#NUM!` (and IronCalc matched Excel), a four-function cluster treated boolean cells as plain numbers, four Treasury-bill functions used a wrong day count, and `DURATION`/`MDURATION` diverged from documented Macaulay duration (which IronCalc reproduced to 1e-9).

Concordance is not accuracy. Only beneath the matrix does the aggregate belong. On the 1,311 numerically-comparable cases the engines matched on 1,273 — **97.1%** (935 exact, 338 within tolerance); across all 1,358 both-value cases, 93.7% (agreement.json). Two engines reading the same ECMA-376 and vendor prose can converge on the same wrong behavior, so agreement bounds the *disagreement* count from below, not the *error* count from above. LibreOffice is a reference peer, not ground truth; this paper never claims "IronCalc is 97.1% correct." The only correctness statements it makes are the 72 arbitrated per-case verdicts, each traceable to Excel documentation.

1.9.2 The confusion matrix as the write-reliability gate

The oracle is not only after-the-fact validation; its confusion matrix is *load-bearing for write safety*, and wiring it into the write path answers the sharpest systems-security finding of the re-review. That finding: the surgical writer persists engine-computed cached values on the sheets it rewrites, and the oracle itself found the engine computes some functions wrong — which reopens the value-corruption class the boundary exists to close and appears to contradict the engine-independence of the fidelity property. The fix folds the oracle directly into the write path. Every IronCalc-wrong verdict above names a function; a real `xlq apply` refuses to commit (`coverage_unreliable`, `oracle_divergent_functions: [...]`) when any cell in the edit's affected dependency graph uses a function on that divergence list — CONVERT, TRIM, ROW,

SUMPRODUCT, MAXA/MINA/STDEVA, SECOND, PRICE, PERCENTRANK.EXC. So xlq never persists a computed cache it has documented reason to believe is wrong. The gate is scoped precisely: it fires only on *engine-computed* formula caches, because those are the values the oracle can indict — a literal `set_cell` writes exactly what the user typed and is engine-independent, so it is never gated. This also retires the re-review's "loosely coupled" critique: the differential oracle and the write boundary are no longer two adjacent components but one mechanism, the confusion matrix deciding which computed values the writer is permitted to persist.

1.9.3 E3: an independent financial cross-check retires circularity for two verdicts and downgrades a third

The largest fault asymmetry — 13 LibreOffice-wrong to 3 IronCalc-wrong on the financial functions — rested, in the earlier work, entirely on author-written Python reimplementations of the day-count and pricing formulas: a single in-house third implementation, corroborating but not independent. To retire that circularity, we cross-checked the financial verdicts against an *independent* reference — `numpy-financial` plus documented Treasury-bill formulas, implemented separately from the triage reimplementation (benchmarks/financial_crosscheck.json). The result both strengthens and corrects the oracle, and we report the correction as prominently as the confirmations:

Function (case)	Independent reference	IronCalc	LibreOffice	Original triage	Cross-ch
TBILLPRICE (184d, 0.045)	97.7	97.7	97.7375	LibreOffice wrong (DSM=181)	**CONFI**
TBILLYIELD (184d, 98.5)	0.0297947	0.0297947	0.0302886	LibreOffice wrong (DSM=181)	**CONFI**
TBILLEQ (184d, 0.045)	0.0466813	0.0466902	0.0466812	LibreOffice wrong	**CONTI**
RATE(10,−100,1000)	−9.49e-11	−3.35e-11	6.12e-11	Spec-ambiguous	CONFIR

Table 8. Independent financial cross-check (financial_crosscheck.json).

For **TBILLPRICE** and **TBILLYIELD**, the independent reference reproduces IronCalc's value to the reported precision and confirms that LibreOffice used a 181-day count where the documented convention gives 184. Those two verdicts no longer rest on our own reimplementation; the circularity objection is retired for them. For **TBILLEQ**, the cross-check *contradicts the original triage*: the independent bond-equivalent form matches LibreOffice's `0.0466812`, not IronCalc's `0.0466902`. We therefore **downgrade that verdict from "LibreOffice wrong" to UNDECIDABLE-HERE pending a licensed Excel** — the independent bond-equivalent formula may itself differ from Excel's exact algorithm, so neither engine can be faulted on the present evidence. We state this as a *strength of the method, not a defect hidden*: an oracle whose own independent check can overturn one of its verdicts is an oracle that is falsifiable, and the TBILLEQ downgrade is exactly the kind of self-correction a hand-adjudicated triage most needs. It also sharpens the honest scope of the financial row: two of its verdicts are now independently corroborated, one is withdrawn, and the cleanest remaining strengthening — the one this cross-check cannot supply — is Excel-the-binary as a third differential peer, unavailable on the Linux host and stated as future work.

1.9.4 What is, and is not, new here

None of the *method* is new: generating inputs in a well-defined subset and cross-voting independent implementations is the differential-testing playbook [Yang et al. 2011], and arbitrating divergences against a prose standard is what CSmith does against the C standard. The contribution claimed for C3 is (a) the *domain* — spreadsheet formula engines, which unlike compilers [Yang et al. 2011], SQL engines [Rigger and Su 2020], and JS engines [Park et al. 2021; SmartOracle 2026] have no mechanized specification to build a stronger oracle from; (b) an explicit, artifact-backed arbitration protocol for that spec-less setting, now including an independent cross-check that overturned one of its own verdicts; and (c) the concrete bugs it surfaced in both engines. Two blind spots are named rather than smoothed: the 1,659 cases are hand-authored at roughly 3.4 per function, so 85 disagreements is a *lower bound* on divergence; and the strongest arbiter, Excel-the-binary, entered only as documentation because no licensed Excel was available. This is what earns the runtime's honesty: the engine's outputs — and the dry-run predictions built on them — are trusted because they

are differentially validated with documentation arbitration and, for the financial functions, an independent cross-check, not because IronCalc is assumed correct.

1.10 9. AXLE-bench: Measuring the Omitted Dimension

Prior agent benchmarks score whether the task cells came out right; none reports whether the file survived. AXLE-bench (Agent-safe eXcel Layer Evaluation) is a consolidated suite that makes the fidelity gap of §2 a first-class measurement axis, beside the four dimensions on which a spreadsheet tool is ordinarily judged. Following OS-Harm's taxonomy-before-benchmark discipline [OS-Harm 2025], the axes are fixed first and every tool instantiated against them symmetrically. Every number traces to a JSON artifact in `benchmarks/`, and one meta-runner reproduces them on a single commodity machine (Intel i7-8700K, 12 logical cores, Linux). The five axes are: **Correctness** — the differential oracle and its disagreement triage, headline being the confusion matrix of §8 (now load-bearing for write safety: it gates which engine-computed caches a real apply may persist), not the 97.1% concordance; **Fidelity** — the omitted dimension, now measured two ways: the whole-file re-save tiers of §2 (101,961 blanked by openpyxl, 90,448 drifted by LibreOffice) *and* the per-file surgical-preservation results of §3 (E1) on real charts/pivots/VBA, plus the interventional case study of §5 (E2) and the structural reference-shift envelope of §4 (E4, the controlled 6/6-vs-0/6; E5, the 231-workbook corpus at 78.8% safe, 182/182 round-trip-correct, zero silent corruptions); **Efficiency** — median-of-3 warm wall time on the ~100k-formula fixture (`xlq calc`, which loads, fully recalculates, audits stored-vs-recomputed, hashes, and reports, at 1.264 s; isolated engine load 0.599 s; `libreoffice --convert-to` 1.494 s; openpyxl `load_workbook` 0.875 s with recalc n/a — it has no formula engine); **Agent-ergonomics** — the census token ratio (999 B vs 7,239,303 B, 7,246.5×); and **Catalog** — the three-number coverage (522/522 recognized, 505 evaluable, 17 policy-limited).

Instrument validation as a co-headline. The measuring instruments are themselves fallible, so their limits are reported beside their outputs. The correctness axis has no live Excel oracle; the 1,659 cases are hand-authored at ~3.4 per function, making the disagreement rate a *lower bound*; and, new in this version, an independent financial cross-check *overturned one of the oracle's own verdicts* (TBILLEQ, §8) — the sharpest instrument-validation point available, because the tool caught its own error. The fidelity axis compares zip central-directory part names, sizes, and CRC32s, a byte-level basis that is exactly why cosmetic drift (LibreOffice's 90,448) has to be separated from consumer-breaking loss (openpyxl's 101,961) by hand, and why the surgical writer's "byte-identical elsewhere" property is a meaningful signal rather than noise. An earlier `xlq diff` once reported one change on an openpyxl re-save while 442 formula results had silently vanished — the exact damage the boundary exists to catch, hiding inside the measuring tool — which is why the `cached_value` change-kind exists. We report these blind spots because the property the suite is built to measure is a tool's willingness to report its own.

The comparison, in one claim. No other tool in the matrix wraps agent writes in preview → apply → receipt → revision, and none reports its own per-artifact blind spots — not openpyxl, not LibreOffice, not the higher-star MCP and CLI wrappers built on openpyxl underneath (verified against each tool's documentation, `docs/BASELINE.md`). And now, crucially, no other tool leaves the parts an edit did not touch byte-identical: the surgical writer is alone in the "fidelity by construction" cell, which the whole-file substrates cannot occupy however careful their save. That gap — not raw speed — is the dimension AXLE-bench exists to measure and to keep honest.

1.11 10. Related Work

We place related work after the mechanism so the reader meets the boundary before the literature it draws on. We did not invent receipts, transactional sandboxing, or differential testing. The claimed contribution is narrower: making an agent *write* boundary aware of its target's *format fidelity* — surgical, with a fidelity property enforced and checked per apply, and demonstrated under a real agent — and validating a formula engine by documentation-arbitered differential testing in a domain with no mechanized specification.

Runtime enforcement for LLM agents. A 2024–26 wave established that deterministic runtime enforcement outperforms prompt- and heuristic-level defenses. Progent [Progent 2025] reports 0% attack success under deterministic policies against 39.9% for prompt-based mitigation; AgentSpec [AgentSpec 2025] compiles

runtime rules from a DSL; MiniScope [MiniScope 2025] derives least-privilege OAuth scopes; IsolateGPT [IsolateGPT 2024] enforces process isolation; a transactional sub-line makes writes reversible — ACID-snapshot sandboxing [Fault-Tolerant Sandboxing 2025] reports 100% filesystem rollback, and semantic/compensating-transaction systems [Cordon 2025; SAFEFLOW 2025] extend it to multi-step actions. We adopt the shared finding that a runtime is the only place a guarantee can live, and we scope our boundary the way this wave scopes its own — non-bypassable *within the harness*, not against an agent handed raw capability (§6). The shared blind spot we differentiate against is that every one of these systems is *format-blind*: it can gate a filesystem write or a shell command, but none understands the artifact it guards, so none can answer *will this edit corrupt the workbook's semantics?* Our write layer is not novel for being atomic; its differentiators are that it is *format-aware and built* — the surgical writer preserves parts by construction (§3), and the dry-run's prediction is *semantic* (affected cells, new formula errors, watched values), which requires the format-aware engine the enforcement wave does not carry.

Receipts and provenance. Cryptographic action receipts and tamper-evident logs are established: Notarized Agents [Notarized Agents 2026] issues receiver-attested receipts for agent actions; forward-secure MAC logging [Nitro 2025], Certificate Transparency [Certificate Transparency 2022], and W3C PROV / PROV-AGENT [Moreau et al. 2013] supply the provenance ancestry. Our receipt journal is a domain instantiation — a single-writer local sidecar with a hash chain and out-of-band edit-adoption semantics, targeting custody of a file a human co-edits — now *exercised* by the built `apply` path (a receipt per revision in §3–§5) rather than specified only.

Differential testing. The oracle-construction menu is mature: cross-vote generation in a well-defined subset [Yang et al. 2011], metamorphic EMI [Le et al. 2014], a tool-built reference evaluator as arbiter [Rigger and Su 2020], a mechanized specification as the (N+1)-th implementation [Park et al. 2021], and LLM-mediated triage against a normative spec [SmartOracle 2026], with Liu et al. [Liu et al. 2024] contributing the three-way verdict vocabulary our triage adopts. Each spec-as-arbiter technique presupposes a mechanized or normative specification the spreadsheet domain lacks. Our instantiation contributes documentation-arbitered triage over 1,659 curated cases, now with an independent financial cross-check that corroborated two verdicts and overturned a third (§8); while cross-engine spreadsheet-accuracy comparisons against certified reference values exist [Almiron et al. 2010; McCullough 2008], no prior work applies generative differential testing with automated documentation-grounded triage to spreadsheet formula engines.

Spreadsheet research and LLM spreadsheet agents. The motivation bedrock is replicated error-pervasiveness work: Panko [Panko 2008] found errors in 94% of 88 audited sheets, and Hermans and Murphy-Hill [Hermans and Murphy-Hill 2015] found Excel errors in 24% of formula-bearing Enron sheets, with 75% using only the top-15 functions. The EUSES corpus [Fisher and Rothermel 2005] and ExceLint [Barowy et al. 2018] are classical ancestors of the structural census. On the agent side, SheetCopilot [Li et al. 2023], SheetAgent [SheetAgent 2025], SpreadsheetBench [SpreadsheetBench 2024; SpreadsheetBench 2026], and Finch [Finch 2025] all score task-answer cells by exact match; none measures file corruption or preservation of untouched content, SheetCopilot's checker skips unflagged properties by design, and only OS-Harm [OS-Harm 2025] measures agent side effects at all — and not file fidelity. The census occupies a third representation design point distinct from model-side encoders [Wang et al. 2021] and prompt-side lossy compression for understanding [Dong et al. 2024]: a tool-side, structure-only summary for mutation safety.

Structural spreadsheet editing. The structural-edit primitive of §4 sits in a gap three prior lines leave open, each missing on a different axis. Excelsior [Excelsior 2008] performs the same insert/delete/resize/move operations and shifts the dependent formulae correctly, but by discovering a high-level model and *regenerating* the file — explicitly discarding "cell formats and colours, input menus, and charts," precisely the fidelity our surgery keeps. TACO [TACO 2023] formalizes the relative/absolute reference-shift algebra our re-encodes, but maintains it over an *in-memory* dependency graph for recalculation and never edits the file, a chart, or a defined name. PPTArena / PPTPilot [PPTArena 2025] brings fidelity-aware, deterministic OOXML patching to agentic editing — but for PowerPoint, which has *no* formula-reference-shift problem, and measures preservation with a VLM-judge over screenshots rather than a checkable invariant. None occupies the correct-plus-byte-minimal-plus-fidelity-preserving cell §4 targets, and none carries a proof that the output re-opens and computes; the contribution is the uniform algebra under a machine-checkable minimal-patch invariant with a never-silently-wrong refusal, not the reference algebra itself, which is Excel's.

Pista — the closest prior. Pista [Pista 2026], with Gulwani among its authors, is the nearest system: step-level human oversight of a spreadsheet agent with model-generated explanations of each action. Its guarantees are, by its authors' account, "solely system prompt instructions," it neither verifies that an explanation matches the executed snippet nor measures whether the file survived, and it wants "programmatic controls" as future work. This paper walks through that door with a machine-checked, format-aware boundary that operates unattended, and it supplies what a synchronous human auditor cannot certify: that the artifact the user is accountable for is still byte-for-byte intact where the edit did not reach.

1.12 11. Discussion

1.12.1 The journey: from a read-only boundary that was rejected to a built write path with an enforced fidelity property

The honest history of this work is its strongest argument. An earlier version diagnosed the fidelity gap correctly, built a read-only inspector that could observe the #22044 damage, and specified — but did not build — the write path. A unanimous program-committee rejection (mean −1.83) named three fatals, and each is now backed by a built artifact rather than a specification. The "enforcement boundary" was unbuilt, read-only, enforcement-by-absence of a write path; it is now a built surgical write primitive with a fidelity property that holds by construction *and is enforced and checked per apply* — each write re-loading its own output to confirm the predicted edit landed and byte-diffing every non-edited part, aborting with `fidelity_violation` on any change before committing (§3, 128 tests) — so the word "enforcement" is earned rather than reached for. There was no threat model and no account of opt-in; there is now an explicit one whose non-bypassability was *tested in a single adversarial trial* in which an adversarial agent could not bypass the confined harness — forgery blocked, and the one destruction hole it found closed by a read-only authoritative store and a receipt-gated broker — rather than merely argued (§6). No agent was ever run; in a single observed trajectory a real LLM agent has now reproduced the #22044 corruption live through the status-quo tool — including the false-success failure where it reported success on a non-reloadable file — and been shown mechanically unable to commit it when confined to xlq (§5). We keep these claims at the scale of their evidence: a case study of two tasks, one adversarial trial, four fidelity fixtures. The property that ties them together is that the write does *not* rewrite the whole file: because the surgical writer copies un-edited parts byte-for-byte and the apply refuses to commit if any un-edited part moved, an agent confined to it *cannot* reach the failure mode, and the reason is mechanical rather than a matter of the agent's care.

1.12.2 The bug in our own tool

The finding that most validated the thesis was in our own hands. On a re-saved copy of `branch-consolidation.xlsx`, `xlq diff` once reported *one change* while 442 formula results had silently vanished — an openpyxl round-trip had blanked every `<v/>` cache, yet the diff, comparing formatted display strings, saw only the single cell whose rendered text moved and pronounced the file intact. The tool built to make silent corruption visible was itself rendering corruption invisible. Surface verification, not code review, caught it — running the tool end to end and reading its output against the file, the same discipline the boundary imposes on an agent — and the fix added the recompute-aware `cached_value` change-kind. The TBILLEQ downgrade in §8 is the same discipline applied to the oracle: an independent cross-check overturning one of our own verdicts is the strongest evidence that the method is falsifiable rather than self-flattering.

1.12.3 Load-bearing assumptions, stated plainly

The fidelity property is by construction, enforced and checked per apply, and corroborated by measurement. Each write re-loads its own output and confirms the predicted edit landed, and byte-diffs every non-edited part and aborts on any change — so both the edited half ("the written file verifiably computes it") and the byte-identical-elsewhere half ("the un-edited parts are verifiably untouched") are checked at write time, not only measured on the fixtures. The single residual that still rests on the writer's control flow is *minimality* — that a rewritten sheet had to be rewritten — which the byte-diff cannot establish, and a reviewer should read that one half as the assertion it is (§3). *IronCalc is still used for the dry-run prediction* — but it never touches the written file; the write is pure zip surgery, so engine-fidelity limits affect the *prediction quality* (flagged

by coverage-honesty and the coverage gate), not the preservation guarantee, which is engine-independent — and, for computed caches, the engine's reach is bounded further by the oracle write-gate, which refuses to persist an engine-computed value whose function the differential oracle found divergent (§8). *Engine trust is differential-earned, not assumed*, and for the financial functions now independently cross-checked, with the one overturned verdict reported. *Non-bypassability is a property of the harness*, which xlq enables but does not itself enforce; we claim it within the harness, tested it in one adversarial trial, and say exactly where it stops.

1.13 12. Limitations

Each limitation is tied to a specific mechanism; where it bounds a headline number, we name the number and direction.

Small fidelity corpus; E1 is four hand-picked fixtures. The per-file preservation results (§3) are a demonstration on four workbooks chosen to carry charts, pivots, and VBA, not a corpus-wide rate. The byte-diff establishes only the byte-identical-elsewhere half; the necessity of each rewritten sheet rests on the writer's construction. The LibreOffice column is a `--convert-to` re-save proxy, an *upper bound on re-save churn*, not a like-for-like single-cell edit.

The agent case study is two tasks, one model, one trajectory per arm. E2 (§5) is an interventional case study that demonstrates the harm and its prevention exist and are reproducible under a real agent; it is not a corruption rate and implies no generality. Both tasks are single `set_cell` edits — the structural-edit primitive of §4 (row/column insert and delete) is built but unexercised under the case study, and formula rewrites and sheet add are neither — the ”agent” is a bounded LLM subagent rather than a full autonomous IDE agent, one trajectory was observed per arm rather than a distribution over agent skill, and the openpyxl arm is the *charitable* openpyxl (the realistic `keep_vba=False` default is strictly worse), so the status-quo numbers understate the harm.

The dry-run's prediction depends on IronCalc. The write preserves parts regardless of the engine, but the *predicted* affected cells, new errors, and coverage flag are only as good as the vendored engine; coverage-honesty flags where they cannot be trusted, but a wrong-but-confident engine value would produce a wrong-but-confident prediction that the coverage gate does not catch.

The oracle is hand-authored and lacks a live Excel; one financial verdict is now undecidable. The 1,659 cases at ~3.4 per function make the disagreement rate a lower bound, adjudication was by hand without a second blind rater, and Excel entered only as documentation. The independent cross-check retired circularity for TBILLPRICE and TBILLYIELD but **downgraded TBILLEQ to undecidable pending a licensed Excel** — the single clearest remaining strengthening of the oracle. Two residual weaknesses persist: the suite is a hand-authored conformance set, not a generator (so coverage of each function's input space is shallow), and E3's cross-check still reimplements the same documented formulas rather than consulting a fully external binary — a licensed Excel as a third differential peer is the strengthening neither the cross-check nor the divergence gate can supply on the Linux host.

Non-bypassability was tested in one adversarial trial (n = 1), and is still scoped to the harness. In that single trial an adversarial agent could not bypass the confined harness — forgery blocked, and the one destruction hole found and closed with a read-only authoritative store and a receipt-gated broker (§6) — but one trial is a probe, not a bypass-resistance proof, and xlq still does not defend a workbook against an agent granted an unconfined shell with an interpreter and a writable authoritative file; that is the harness's responsibility, stated as a scoping assumption, not a guarantee.

Structural edits are now built, but within an honest envelope. Row and column insert and delete — the operation the earlier version of this work named as its key open problem — are implemented (§4), but three edges bound them, and where one bounds a headline number we name it. First, array formulas (which Excel itself forbids splitting across an insert line) and workbooks carrying table parts are *refused*, not shifted: on the 231-workbook corpus this is 21.2% of files (arrays 20.3%, tables 0.9%), declined with a truthful reason rather than silently corrupted, and named as future work. Second, the minimal-patch purity of §3 is *relaxed* for shared formulas, which are materialized into explicit per-cell formula bodies before shifting (what Excel

does internally) — a deliberate trade of byte-minimality on those cells for correctness on the dominant construct, which is precisely what lifted the safe envelope from 22.5% to 78.8%. Third, the end-to-end evaluation exercises one operation family, insert and delete of a row; column operations share the same and are unit-tested (including the whole-column cases the adversarial review surfaced) but are not in the end-to-end corpus table, while sheet add/remove and range moves remain unbuilt. The structural primitive is also unexercised under the agent case study (§5) and outside the non-bypassability trial (§6), both of which stay scoped to value and formula edits.

The census drops phonetic runs. The structural census and the engine's import path do not preserve furigana `rPh` runs, so `PHONETIC` evaluates against base text and carries no oracle signal (LibreOffice does not implement it either).

1.14 13. Conclusion

This paper set out to give an LLM agent a way to *edit* a spreadsheet that it cannot use to corrupt one, for an artifact class whose owner is accountable for every cell. The core is a surgical transactional write with a property that holds by construction and is enforced on every write: after an edit, every OOXML part without a changed cell is byte-identical to the input, because the writer copies those parts verbatim and re-serializes only the sheet parts an operation touches — and because each apply byte-diffs the un-edited parts and aborts on any change before it commits. Four things were learned in building and testing it. First, the property is real and it is the escape from the fidelity gap of §2: the only way to keep byte-identity as an integrity signal is to stop rewriting the whole file, which no whole-file substrate does. Second, the property survives contact with a real agent: in an interventional case study of two tasks, a status-quo agent reproduced the #22044 corruption live in one observed trajectory and reported success on a non-reloadable file, while an agent confined to xlq made the identical edit with charts, pivot, and VBA byte-identical and left a receipt. Third, the boundary's honesty is machine-checkable and falsifiable: the runtime declares per file what it cannot evaluate, the write is refused when its prediction is unreliable, and an independent financial cross-check corroborated two of the engine's verdicts and overturned a third — which we report rather than hide. Fourth, an evaluation can measure fidelity; AXLE-bench adds the axis the field's benchmarks omit. We claim no new cryptography, transaction model, or differential-testing technique — receipts, transactional rollback, and cross-engine differential testing are established — no formal proof, and no result beyond demonstration scale. The contribution is to fold the artifact's format fidelity into an agent write boundary, surgically and with an enforced-and-checked fidelity property, and to demonstrate under a real agent — at the scale of a case study — that an agent confined to it cannot commit the corruption that shipped in production, for the in-place value and formula edits it covers — and, in a sixth contribution, for row and column insert and delete, the structural operations the field and this work's own earlier scope named as the hard open problem, resolved by a uniform reference-shift algebra under a machine-checkable minimal-patch invariant, with an honest refusal for the constructs (array formulas, tables) it does not yet shift and a real-corpus envelope that edits 78.8% safely and declines the rest with zero silent corruptions. The earlier version of this work could observe that corruption; this one prevents it, within the scope it honestly claims.

1.15 References

Runtime enforcement for LLM agents

- **[Progent 2025]** Progent: Programmable Privilege Control for LLM Agents. 2025.
- **[AgentSpec 2025]** AgentSpec: Runtime Enforcement of LLM-Agent Safety via a Domain-Specific Rule Language. 2025.
- **[MiniScope 2025]** MiniScope: Least-Privilege OAuth Scope Derivation for Agents via Integer Programming. 2025.
- **[IsolateGPT 2024]** IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. 2024.
- **[Fault-Tolerant Sandboxing 2025]** Fault-Tolerant Sandboxing for LLM Agents: ACID Snapshots with 100% Filesystem Rollback. 2025.
- **[Cordon 2025]** Cordon: Semantic Transactions for Multi-Step Agent Actions. 2025.

- [**SAFEFLOW 2025**] SAFEFLOW: Compensating-Transaction Safety for Agent Workflows. 2025.

Receipts and provenance

- [**Notarized Agents 2026**] Notarized Agents: Receiver-Attested Cryptographic Receipts for Agent Actions. 2026.
- [**Nitro 2025**] Forward-Secure MAC Logging for Tamper-Evident Audit Trails. ACM CCS, 2025.
- [**Certificate Transparency 2022**] SoK: Certificate Transparency and Merkle-Proof Audit Logs. IEEE S&P, 2022.
- [**Moreau et al. 2013**] L. Moreau et al. The PROV Data Model (W3C PROV) and PROV-AGENT. W3C Recommendation, 2013.

Differential testing

- [**Yang et al. 2011**] X. Yang, Y. Chen, E. Eide, J. Regehr. Finding and Understanding Bugs in C Compilers (CSmith). PLDI, 2011.
- [**Le et al. 2014**] V. Le, M. Afshari, Z. Su. Compiler Validation via Equivalence Modulo Inputs (EMI). PLDI, 2014.
- [**Rigger and Su 2020**] M. Rigger, Z. Su. Testing Database Engines via Pivoted Query Synthesis (PQS/SQLancer) and NoREC. OSDI / ESEC-FSE, 2020.
- [**Park et al. 2021**] J. Park et al. JEST: N+1-Version Differential Testing of JavaScript Engines with a Mechanized Specification. ICSE, 2021.
- [**SmartOracle 2026**] SmartOracle: LLM-Mediated Triage of JavaScript-Engine Disagreements Against the ECMAScript Specification. 2026.
- [**Liu et al. 2024**] Liu et al. Specification-Based Differential Testing of SQL Engines with a Three-Way Verdict. 2024.
- [**Almiron et al. 2010**] M. Almiron et al. On the Numerical Accuracy of Spreadsheets. Journal of Statistical Software, 2010.
- [**McCullough 2008**] B. D. McCullough. Assessing the Reliability of Statistical Software in Spreadsheets. 2008.

Spreadsheet research and LLM spreadsheet agents

- [**Panko 2008**] R. R. Panko. What We Know About Spreadsheet Errors. 2008.
- [**Hermans and Murphy-Hill 2015**] F. Hermans, E. Murphy-Hill. Enron's Spreadsheets and Related Emails. ICSE, 2015.
- [**Fisher and Rothermel 2005**] M. Fisher, G. Rothermel. The EUSES Spreadsheet Corpus. 2005.
- [**Barowy et al. 2018**] D. W. Barowy, E. D. Berger, B. Zorn. ExceLint: Automatically Finding Spreadsheet Formula Errors. OOPSLA, 2018.
- [**Li et al. 2023**] H. Li et al. SheetCopilot. NeurIPS, 2023.
- [**SheetAgent 2025**] SheetAgent: A Generalist Agent for Spreadsheet Reasoning and Manipulation. WWW, 2025.
- [**SpreadsheetBench 2024**] SpreadsheetBench: Towards Challenging Real-World Spreadsheet Manipulation. NeurIPS D&B, 2024.
- [**SpreadsheetBench 2026**] SpreadsheetBench 2: Successor Benchmark. 2026.
- [**Finch 2025**] Finch: Evaluating LLM Agents on Real-World Finance Spreadsheet Workflows. 2025.
- [**OS-Harm 2025**] OS-Harm: Measuring Harmful Side Effects of Computer-Use Agents. NeurIPS, 2025.
- [**Wang et al. 2021**] Z. Wang et al. TUTA: Tree-Based Transformers for Generally Structured Table Pre-Training. KDD, 2021.
- [**Dong et al. 2024**] H. Dong et al. SpreadsheetLLM: Encoding Spreadsheets for Large Language Models. 2024.
- [**Pista 2026**] Pista: Step-Level Human Oversight of Spreadsheet Agents. 2026.

Structural spreadsheet editing

- [**Excelsior 2008**] J. Paine, E. Tek, D. Williamson. Rapid Spreadsheet Reshaping with Excelsior. EuSpRIG, 2008. (arXiv:0803.0163)

- [**TACO 2023**] D. Tang et al. TACO: Efficient and Compact Spreadsheet Formula Graphs. 2023. (arXiv:2302.05482)
- [**PPTArena 2025**] N. Ofengenden et al. PPTArena / PPTPilot: Fidelity-Aware Agentic OOXML (PowerPoint) Editing. 2025. (arXiv:2512.03042)

Standards and artifacts

- [**claude-code#22044**] Anthropic. claude-code issue #22044: xlsx agent skill corrupts financial models (charts, pivots, VBA) via openpyxl. GitHub, 2025.
- **ECMA-376 / ISO-IEC 29500.** Office Open XML File Formats. (Standardizes the OOXML container, not formula numerical semantics.)
- **OpenFormula.** OpenDocument Formula Specification (ODF). OASIS. (Normative for ODF, not Excel's dialect.)